Curating a Robust LLM Pool: A Two-Stage Stochastic Optimization Approach for Cost-Constrained LLM Routing

Yu Hin Liang, Jingwen Yang
INDENG 164, Spring 2026
May 8, 2026

Abstract

LLM routing asks which model to call on each user prompt. We cast this problem for an agentic AI company as a two-stage stochastic mixed-integer program. The first stage selects a small *pool* of models to maintain in production, and the second stage routes each realised prompt to one of the selected models. Sample-average approximation on `routerbench` (240 prompts, 32 models, four benchmark domains) produces a tractable MILP. We add four realistic deployment constraints (a pool-size cap, a storage budget, a per-prompt fairness slack, and provider rules) and a distributionally robust extension that hedges against shifts in the prompt mixture. The resulting policy traces a smooth Pareto frontier on the cost-quality plane. At a budget of \$0.001 per prompt the policy reaches a weighted score of 0.967, which already exceeds the best single model GPT-5 by eight percentage points at fifty times lower cost. Saturating the budget at \$0.005 per prompt drives the score to the per-prompt oracle of 0.983, a thirteen-fold cost reduction relative to running GPT-5 on every prompt. A pool of $K = 6$ already captures 99.7% of the achievable quality, and a $\pm 70\%$ shift in benchmark proportions costs at most 1.2 percentage points of worst-case quality.

1 Introduction

We adopt the role of an operations-research scientist at *CodePilot AI*, an agentic AI company that ships a Cursor-style coding assistant and a tutoring sidebar for STEM students. Its workload spans code, mathematics, and knowledge questions. For every prompt the company picks one of about thirty candidate LLMs, ranging from GPT-5 at \$0.04 per prompt down to free open-source 9 B-parameter backbones. Choosing well, prompt by prompt, is the difference between profit and loss.

Two decisions live on different time scales. *Pool curation* selects which models to maintain in production and binds for weeks because each model carries integration, monitoring, and storage cost. *Routing* selects which model to call on a given prompt and repeats over a stream of stochastic future requests. The best pool depends on the routing policy that will operate on it, and the best policy in turn depends on which models the company actually maintains. This coupling is the structure of a two-stage stochastic program with recourse. The pool y is first-stage and is committed before the prompt distribution is realised, while the routing x_ξ is second-stage and is made after the prompt is observed. The prompt distribution $\mathbb{P}$ is unknown and we observe only $|\mathcal{P}| = 240$ samples, so we use sample-average approximation. Model correctness is itself a Bernoulli outcome, which also makes the second-stage policy naturally stochastic.

We study two questions. **RQ1: pool curation.** How many models, and which ones, should the company maintain under a fixed inference budget, and are there diminishing returns? **RQ2: distributional robustness.** If the prompt mix shifts, how much worst-case quality is lost, and is the optimal pool stable?

2 Data and Notation

The `routerbench` dataset provides binary correctness $Q_{pm} \in \{0, 1\}$ and per-prompt cost $C_{pm} \in [0, \$0.108]$ for 33 models on 240 prompts. The prompts are equally split across four benchmarks (AIME, LCB,

GPQA, MMLU-Pro). One model (deepseek-v3.1-terminus) covers only 180 of the 240 prompts; we discard it and work with the $|\mathcal{M}| = 32$ fully-covered models to avoid ad hoc imputation. Nine of the remaining models have $C_m = 0$ because they are open-source and can be served locally. The most expensive model is gpt-5, at \$0.0425 per prompt and weighted score 0.888 on the dataset. We write $\mathcal{P}_d$ for the prompt subset of domain $d \in \mathcal{D}$, w_p for the SAA prompt weight (uniform $1/|\mathcal{P}|$ at baseline), S_m for the storage footprint of model m in GB (zero for API-only models), and use $(B, K, S^{\max}, \tau, \lambda)$ for the cost budget, pool cap, storage cap, fairness target, and slack price.

3 Optimization Model

Decision variables. The first-stage variables $y_m \in \{0,1\}$, $m \in \mathcal{M}$, encode whether model m is selected into the production pool. The second-stage variables $x_{pm} \in \{0,1\}$ encode the routing decision for each prompt. A non-negative slack $s_p \geq 0$ softens the per-prompt fairness target in cases where the data does not allow it to be met exactly.

Population two-stage program. For a random prompt $\xi \sim \mathbb{P}$, the routing policy is a function $x(\xi)$ from prompts to feasible assignment vectors. The population optimisation is

$$\begin{aligned} \max_{y,\, x(\cdot)} \ & \mathbb{E}_\xi[\textstyle\sum_m Q(\xi, m)\, x_m(\xi)] \quad \text{s.t.} \quad \textstyle\sum_m x_m(\xi) = 1, \ \ x_m(\xi) \leq y_m \ \ \text{a.s.}, \\ & \textstyle\sum_m y_m \leq K, \ \ \sum_m S_m y_m \leq S^{\max}, \ \ \mathbb{E}_\xi[\sum_m C(\xi, m)\, x_m(\xi)] \leq B. \end{aligned} \tag{1}$$

The budget is imposed in expectation rather than per-realisation to match a monthly inference budget; y is first-stage because it is committed before any prompt is seen, and x is second-stage because it depends on ξ.

Sample-average approximation (BC-2SP). The true distribution $\mathbb{P}$ is unknown. We replace it by the empirical distribution on the 240 observed prompts with weights w_p and write $x_{pm} \equiv x_m(\xi = p)$. The deterministic-equivalent MILP is

$$\begin{aligned} \max_{x,y,s} \quad & \sum_{p,m} w_p\, Q_{pm}\, x_{pm} \ - \ \lambda \sum_p w_p\, s_p \ - \ \varepsilon \sum_{p,m} w_p\, C_{pm}\, x_{pm} \\ \text{s.t.} \quad & \textstyle\sum_m x_{pm} = 1, \quad x_{pm} \leq y_m, \quad \sum_m y_m \leq K, \quad \sum_m S_m y_m \leq S^{\max}, \\ & \textstyle\sum_{p,m} w_p\, C_{pm}\, x_{pm} \leq B, \quad \sum_m Q_{pm}\, x_{pm} + s_p \geq \tau, \quad x_{pm}, y_m \in \{0,1\}, \ s_p \geq 0. \end{aligned} \tag{2}$$

The first term is the SAA estimator of expected quality. The second term softly penalises violation of the per-prompt fairness target τ, with slack price λ chosen so that the routing prefers a correct answer whenever one exists. The third term, with $\varepsilon = 10^{-3} \ll \lambda$, is a cost tie-breaker that selects the cheapest among quality-equivalent solutions and removes a non-monotonicity along the upper frontier. Provider rules are encoded by fixing the corresponding y_m.

Distributionally robust extension (DRO BC-2SP). The benchmark mixture $w_d := \Pr[\xi \in \mathcal{P}_d]$ is itself uncertain. We use a relative-box ambiguity set around the empirical proportions $w_d^0 = |\mathcal{P}_d|/|\mathcal{P}|$, namely $\mathcal{U}(\delta) = \{w \in \Delta_{|\mathcal{D}|} : \ |w_d - w_d^0| \leq \delta\, w_d^0 \ \forall d\}$. Let $f_d(x) := \frac{1}{|\mathcal{P}_d|} \sum_{p \in \mathcal{P}_d, m} Q_{pm}\, x_{pm}$ be the conditional expected score on domain d. The DRO objective is $\max_{y,x} \min_{w \in \mathcal{U}(\delta)} \sum_d w_d f_d(x)$. Reparametrising $w_d = w_d^0(1 + e_d)$ with $|e_d| \leq \delta$ and $\sum_d w_d^0 e_d = 0$, the inner LP admits the closed-form dual $\min_w \sum_d w_d f_d(x) = \sum_d w_d^0 f_d(x) - \delta \cdot \min_\lambda \sum_d w_d^0 |f_d(x) - \lambda|$, where the inner minimiser $\lambda^\star$ is the w^0-weighted median of $\{f_d(x)\}$. Linearising the absolute values with $z_d \geq 0$ and a free λ gives the equivalent MILP

$$\max_{x,y,\lambda,z} \ \sum_d w_d^0 f_d(x) \ - \ \delta \sum_d w_d^0 z_d \quad \text{s.t.} \quad z_d \geq f_d(x) - \lambda, \quad z_d \geq \lambda - f_d(x), \quad \forall d \in \mathcal{D}, \tag{3}$$

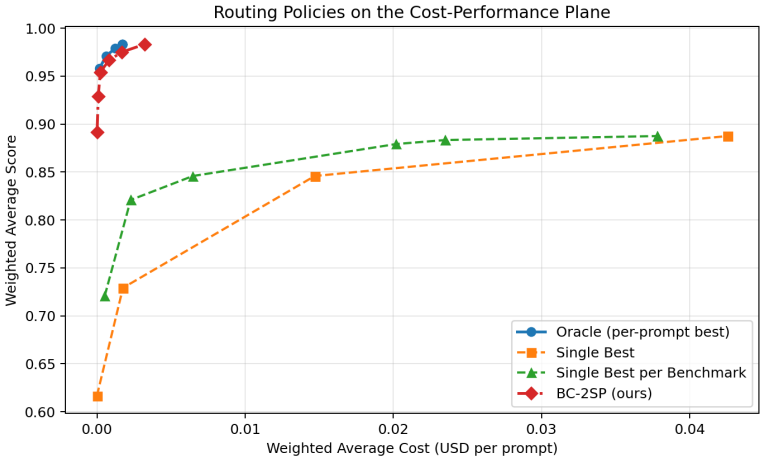


Figure 1: Cost-quality Pareto frontier on `routerbench`. Oracle is the per-prompt unconstrained optimum and serves as an upper bound on any pool-constrained policy. BC-2SP (red) reaches the oracle score with a pool of $K = 8$ and dominates both Single-Best baselines by an order of magnitude in cost.

together with the BC-2SP constraints in (2) and the budget under nominal w. The reformulation adds only $|\mathcal{D}| + 1$ continuous variables and $2|\mathcal{D}|$ inequality constraints, so it is essentially the same size as BC-2SP.

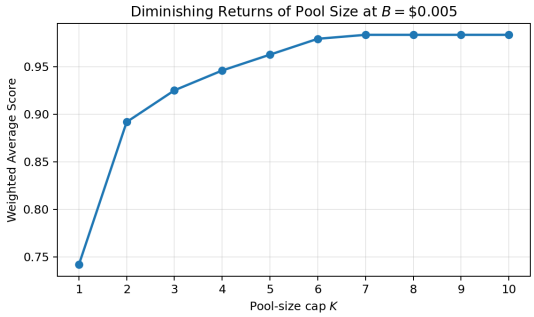
Companion formulations. Three variants share the same constraint set: *QC-2SP* minimises expected cost subject to a target weighted score $Q^{\min}$ (the dual side of the same frontier); *Randomised BC-2SP* relaxes $x_{pm} \in [0, 1]$ into a stochastic policy with $\Pr[\xi = p \mapsto m] = x_{pm}$; *Cascade BC-2SP* pairs (m_1, m_2) per prompt with a zero-cost verifier and escalates only on failure, giving a MILP of size $|\mathcal{P}||\mathcal{M}|^2$.

Solving the MILP. The BC-2SP MILP has 8,224 binary variables and 8,163 linear constraints. We implement BC-2SP, QC-2SP, DRO BC-2SP and the cascade variant in Pyomo with the HiGHS solver; a single instance solves in two to four seconds, and a full eleven-point frontier sweep completes in under one minute. Storage parameters S_m are heuristics from the model name: roughly 14 to 18 GB for 7 to 9 B open-source backbones, 0 GB for provider-hosted APIs, and about 470 GB for the 235 B Qwen mixture-of-experts. The notebook `solution.ipynb` accompanies this report.

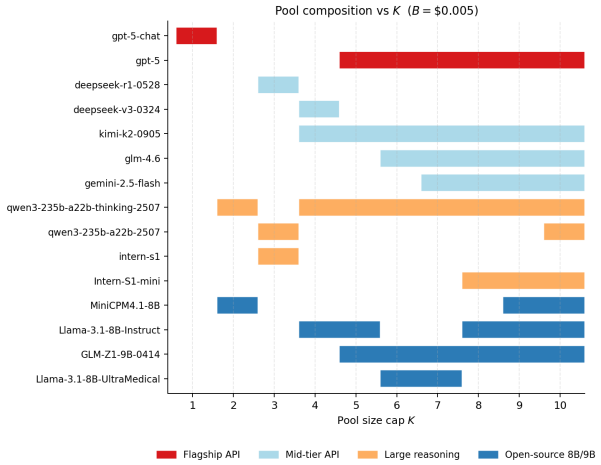
4 Experiments

All experiments use uniform empirical weights $w_p = 1/240$, $\varepsilon = 10^{-3}$, and $\lambda = 1$ unless otherwise stated.

Cost-quality Pareto frontier. We initially adopted the weighted-sum scalarisation min cost $- \alpha \cdot$ score suggested by the starter code, but found that α has no operational meaning and that combining it with a fairness penalty collapses the pool to two or three models at small α. Switching to an explicit budget B removes both issues: the budget has clear monetary meaning, and sweeping it from $\$10^{-5}$ to $\$5 \times 10^{-3}$ at $K = 8$ traces the BC-2SP frontier of Figure 1, which strictly dominates every Single-Best baseline. At $B = \$10^{-5}$ the policy already scores 0.892, marginally above GPT-5 but at four orders of magnitude lower cost, because the optimal pool concentrates on free open-source backbones. At $B = \$10^{-3}$ the score reaches 0.967, which is eight percentage points above GPT-5 at fifty times lower cost. At $B = \$5 \times 10^{-3}$ the policy attains the per-prompt oracle of 0.983 at actual cost \$0.0032 per prompt, a 13.1× cost reduction relative to GPT-5. Beyond this budget the score saturates because no further quality is available in the data.



(a) Diminishing returns of pool cap K at $B = \$0.005$.



(b) Pool composition vs. K. GPT-5 enters at $K = 4$.

Figure 2: Quality saturates at $K = 7$, and a $K = 6$ pool already captures 99.7% of the achievable quality.

RQ1: diminishing returns of pool size. Fixing $B = \$5 \times 10^{-3}$ and sweeping K from 1 to 10 (Figure 2a) shows a fast-then-saturating curve. A single model achieves only 0.742 because no model alone covers all four benchmarks adequately. Adding a second model already lifts the score to 0.892 (a fifteen percentage-point gain). The curve continues to rise but flattens quickly: $K = 4$ reaches 0.946, $K = 6$ reaches 0.979 (99.6% of the $K = 10$ value), and from $K = 7$ on the score is pinned at the oracle 0.983. The operationally relevant message is that six to seven well-chosen models are sufficient regardless of how many candidates the company nominally has access to; further models are pure overhead.

Pool composition and traffic concentration. Figure 2b shows the growth path of the optimal pool. The optimiser first picks complementary free open-source backbones, then admits a flagship model only at $K = 4$, and uses mid-tier APIs to fill in between. In the converged $K = 8$ pool, GPT-5 routes only 1.7% of traffic and serves as a backstop for the four hardest prompts that no smaller model can solve. The free GLM-Z1-9B-0414 alone handles 57.5% of prompts. The remaining traffic is distributed across KIMI-K2-0905 (10.0%), INTERN-S1-MINI (8.3%), LLAMA-3.1-8B-INSTRUCT (7.1%), GLM-4.6 (5.8%), GEMINI-2.5-FLASH (5.4%), and QWEN3-235B-THINKING-2507 (4.2%). The traffic is therefore not concentrated on the strongest single model; the optimal policy is mostly flat with a flagship as backstop.

Per-benchmark breakdown. At $B = \$0.005$, $K = 8$ the policy solves LCB and GPQA perfectly and reaches 0.967 on both AIME and MMLU-PRO. The four remaining prompts are exactly those for which no model in the data produces a correct answer, so the routing matches the per-prompt oracle on every solvable prompt.

RQ2: distributional robustness. Sweeping δ from 0 to 0.7 at $B = \$0.005$, $K = 8$ (Figure 3), the nominal score remains pinned at 0.983. The worst-case score from (3) drops by 0.50, 0.83, and 1.17 percentage points at $\delta = 0.3, 0.5, 0.7$. The DRO pool overlaps with the nominal in seven of eight slots for $\delta \leq 0.5$, so the pool itself is highly stable. The price of distributional robustness on this dataset is therefore small.

Constraint ablations and formulation comparison. Storage caps down to 50 GB are free; tightening to 20 GB costs 1.2 pp because the optimiser falls back from $\sim$8 B backbones onto smaller alternatives. Excluding {GPT-5, GPT-5-CHAT} costs only 0.4 pp (0.983 $\rightarrow$ 0.979 at \$0.0024) since QWEN3-235B-THINKING-2507 and KIMI-K2-0905 absorb the slack; forcing CLAUDE-SONNET-4 preserves the score at 38% more cost. Across formulations, BC-2SP and QC-2SP coincide at the corresponding $(B, Q^{\min})$ pair as a primal-dual pair on the same frontier, the DRO variant trades a small cost increase for δ-protection,

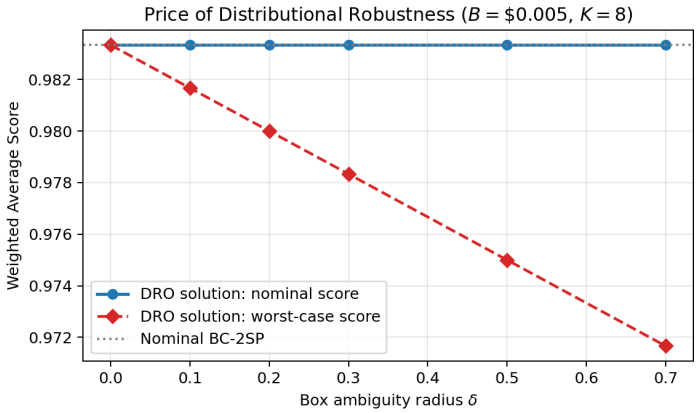


Figure 3: Price of distributional robustness. The worst-case score drops by about 1.2 percentage points at $\delta = 0.7$, while the nominal score stays at the oracle value.

and the randomised and cascade variants give only marginal lift on this binary-score dataset (randomised: +0.35 pp at $B = \$0.001$; cascade: matches BC-2SP at $B = \$0.0005$).

5 Discussion and Recommendations

Answer to RQ1. A pool of $K^\star = 6$ is sufficient and $K = 7$ already saturates at the oracle. The pool the optimiser produces is heterogeneous and not concentrated: GPT-5 routes only 1.7% of traffic, while free open-source backbones together handle about 60%. The dominant strategy on this benchmark is therefore to route most prompts through free models and reserve a single flagship as a backstop for the small set of hard prompts.

Answer to RQ2. The price of distributional robustness is small. The worst-case score drops by at most 1.2 pp when each benchmark proportion swings by $\pm 70\%$, and the optimal pool overlaps in seven of eight slots with the nominal solution for $\delta \leq 0.5$. Since the DRO MILP is the same size as BC-2SP, deploying it by default is essentially free insurance against mix shifts.

Recommendations. Maintain a six-model pool combining a flagship backstop, a mid-tier API, a reasoning-focused model, and two or three free $\sim$8 B open-source backbones (concretely: GPT-5, GEMINI-2.5-FLASH, KIMI-K2-0905, GLM-Z1-9B-0414, INTERN-S1-MINI, LLAMA-3.1-8B-INSTRUCT). Re-solve the SAA MILP whenever cost or quality targets change, treat fairness as soft via s_p, and budget 50 GB of local storage.

Limitations and future work. Binary correctness underuses information; graded scores would let the randomised and cascade variants dominate by a larger margin. Latency and log-loss objectives drop into the SAA MILP without difficulty, and an online-learning loop that updates $\hat{\mathbb{P}}$ from production traffic would close the deployment loop.

Personal takeaway. The striking moment was watching a routing policy thirteen times cheaper than GPT-5 attain its oracle of 0.983. Treating the data as a 240×32 matrix of (Q, C) pairs immediately suggested the two-stage form, and HiGHS solved every instance in seconds, keeping modelling and analysis fast enough to be genuinely fun.

A Detailed numerical results

Formulation	Setting	Score (cost \$/prompt)	|Pool|	Note
BC-2SP	$B = \$0.005$	0.9833 (0.00323)	8	main
QC-2SP	$Q^{\min} = 0.98$	0.9833 (0.00323)	8	dual matches BC-2SP
QC-2SP	$Q^{\min} = 0.95$	0.9500 (0.00019)	8	cheap operating point
QC-2SP	$Q^{\min} = 0.90$	0.9000 (0.0000179)	8	near-free quality
DRO BC-2SP, $\delta = 0.3$	$B = \$0.005$	0.9833 wc 0.978 (0.00500)	8	robust
DRO BC-2SP, $\delta = 0.7$	$B = \$0.005$	0.9833 wc 0.972 (0.00500)	8	wide ambiguity
Randomised BC-2SP	$B = \$0.001$	0.9702 (0.00100)	8	+0.35 pp vs det
Cascade BC-2SP	$B = \$0.0005$	0.9542 (0.00027)	8	matches BC-2SP
Single Best (GPT-5)	(−)	0.8875 (0.04255)	1	best single model
Single Best per Benchmark	(−)	0.8875 (0.03782)	4	one model per domain
Oracle (per-prompt best)	(−)	0.9833 (0.00170)	32	unconstrained upper bound

Table 1: Comparison of formulations and baselines at representative operating points. Cost in USD per prompt. "wc" denotes the worst-case score under the corresponding ambiguity radius.

Setting	Score	Cost (\$/prompt)	|Pool|	Δ vs baseline
BC-2SP free	0.9833	0.00323	8	(−)
no OpenAI (exclude GPT-5, GPT-5-CHAT)	0.9792	0.00240	8	−0.4 pp
must include CLAUDE-SONNET-4	0.9833	0.00444	8	+38% cost
$S^{\max} = 200$ GB	0.9833	0.00495	8	(−)
$S^{\max} = 100$ GB	0.9833	0.00485	8	(−)
$S^{\max} = 50$ GB	0.9833	0.00485	8	(−)
$S^{\max} = 20$ GB	0.9708	0.00441	8	−1.2 pp

Table 2: Constraint ablations at $B = \$0.005$, $K = 8$.

Benchmark	#prompts	Score	Cost (\$)
AIME	60	0.967	0.00592
GPQA	60	1.000	0.00136
LCB	60	1.000	0.00479
MMLU-PRO	60	0.967	0.00086
Aggregate	240	0.983	0.00323

Table 3: Per-benchmark breakdown at $B = \$0.005$, $K = 8$.

δ	nom.	wc	Δ	$\lambda^\star$
0.0	0.983	0.983	0.00 pp	0.00
0.1	0.983	0.982	−0.16 pp	1.00
0.2	0.983	0.980	−0.33 pp	1.00
0.3	0.983	0.978	−0.50 pp	1.00
0.5	0.983	0.975	−0.83 pp	1.00
0.7	0.983	0.972	−1.17 pp	0.97

Table 4: DRO sweep at $B = \$0.005$, $K = 8$. "wc" is the worst-case score from (3).